

API 3 — EXPORT Automation System

Table of Contents

Table of Contents.....	1
1. Executive Summary.....	3
2. Project Overview.....	3
3. System Workflow.....	4
Workflow Narrative.....	4
3.1 Master Workflow — Full Pipeline View.....	5
4. System Architecture.....	6
Architectural Principles.....	6
5. Functional Modules.....	7
5.1 Buyer Search Module.....	7
5.2 Data Extraction Module.....	7
5.3 Email Validation Module.....	7
5.4 AI Email Classification Module.....	7
5.5 Gmail Sender Module.....	8
5.6 Presentation Attachment Module.....	8
5.7 Logging Module.....	8
6. Folder / File Structure.....	8
Key File Responsibilities.....	9
7. Data Model & Database Schema.....	10
7.1 File-Based Schema.....	10
8. Application Interface (Web Routes).....	11
9. Gmail Integration.....	11
9.1 Authentication.....	11
9.2 SMTP Configuration.....	11
9.3 Sending Process.....	12
9.4 Reliability Considerations.....	12
10. Installation & Configuration.....	12
10.1 Requirements.....	12
10.2 Core Libraries.....	12
10.3 Environment Variables (.env).....	12
10.4 Gmail Setup Steps.....	13
11. Execution Guide.....	13
11.1 Step-by-Step Process.....	13
11.2 Expected Output.....	13

12. Core Algorithms.....	13
12.1 Algorithm — Email Extraction & Validation.....	13
12.2 Algorithm — AI Email Classification.....	14
12.3 Algorithm — Send Email Campaign.....	14
13. Error Handling & Limitations.....	14
Known Limitations.....	15
14. Future Improvements.....	15

1. Executive Summary

The EXPORT Automation System (API 3) is a two-stage pipeline built to automate the export marketing workflow for a Singing Bowls export business. It removes the manual effort of prospecting international buyers and pitching them individually by combining multi-platform buyer discovery with automated Gmail-based outreach.

The system operates in two sequential stages:

1. Buyer Discovery — Searches multiple online sources (Google, Facebook, LinkedIn, business directories, and company websites) for individuals and companies expressing interest in or dealing with Singing Bowls, and extracts structured contact records.
2. Automated Outreach — Validates the discovered emails, attaches the company's product presentation, and dispatches personalized emails via Gmail — while preventing duplicate sends and logging every action for reporting.

Core Objectives:

#	Objective	Description
1	Multi-source buyer discovery	Aggregate buyer leads from five+ distinct online channels into a single dataset
2	Structured data capture	Normalize each lead into a consistent schema (name, company, email, website, country, source)
3	Email validation	Filter out malformed, disposable, or unreachable addresses before send
4	Automated Gmail dispatch	Send the company presentation to every valid buyer without manual intervention
5	Duplicate prevention	Ensure no buyer is contacted more than once across runs
6	Logging & reporting	Maintain a persistent record of every discovery and send event for auditing

2. Project Overview

Attribute	Detail
Project Name	API 3 – EXPORT Automation System
Target Product	Singing Bowls (export goods)
Primary Users	Export sales / business development team
Execution Mode	Semi-automated, script-driven (search → review → send)
Delivery Channel	Gmail (SMTP-based dispatch)
Data Store	Flat-file / CSV-based local database
Output Artifact	Company presentation delivered as an email attachment

The system is designed as a lightweight, script-driven pipeline rather than a full SaaS platform — prioritizing speed of iteration and low operational overhead for a small export outreach operation.

3. System Workflow

The end-to-end process runs in two coordinated phases: Buyer Search and Outreach Dispatch. The diagram below traces a single execution cycle.

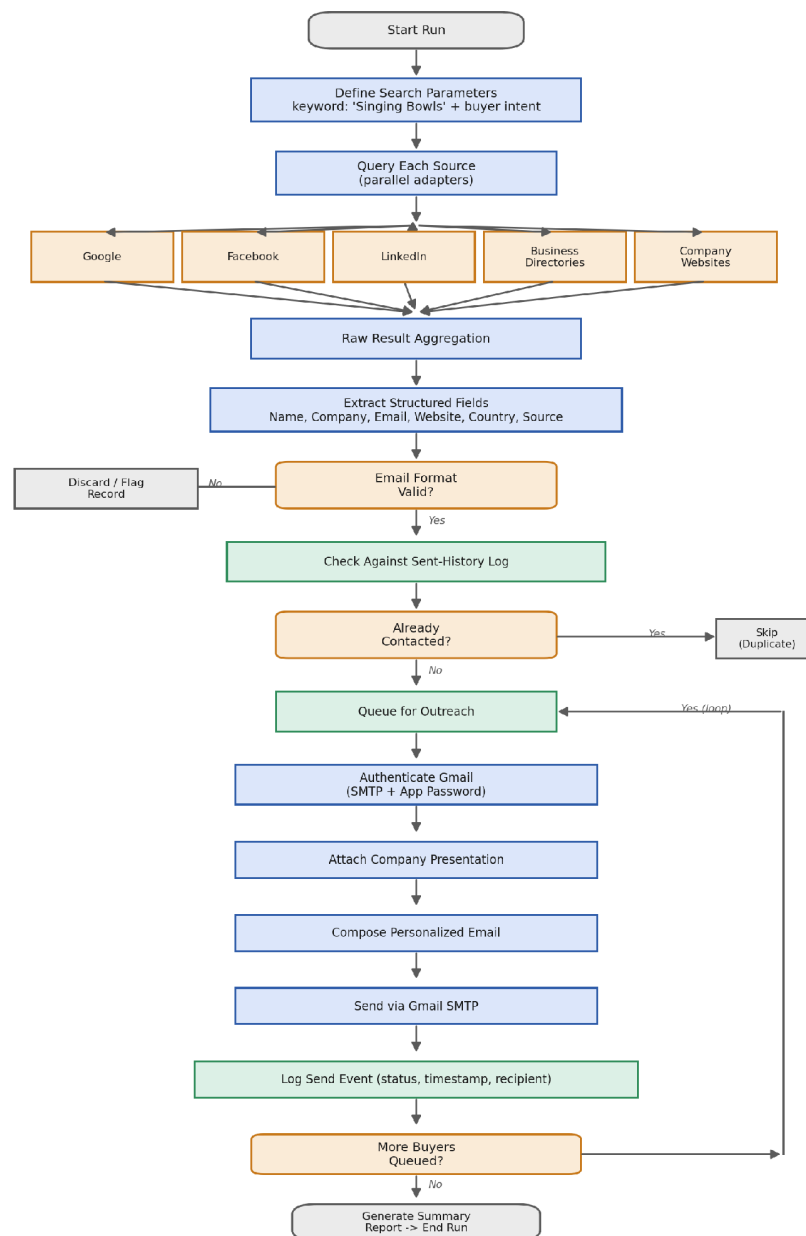


Figure 1 — End-to-end workflow: buyer discovery through outreach dispatch and reporting

Workflow Narrative

3. The operator initiates a run targeting the "Singing Bowls" export niche.
4. The system queries each configured source in turn, collecting raw listing/post/profile data.
5. Every raw result is parsed into a normalized buyer record.
6. Each email is syntactically validated before being accepted into the queue.
7. Valid records are cross-checked against the historical send log to eliminate duplicates.
8. Remaining new buyers are queued, the Gmail session authenticates, the presentation is attached, and a personalized email is dispatched to each.
9. Every send attempt (success or failure) is logged, and a final report summarizes the run.

3.1 Master Workflow — Full Pipeline View

The expanded view below situates the core search-and-send cycle within the wider outreach lifecycle — including the additional lead channels, AI-assisted classification, reporting/analytics, and the automation, settings, and UI subsystems that surround it.

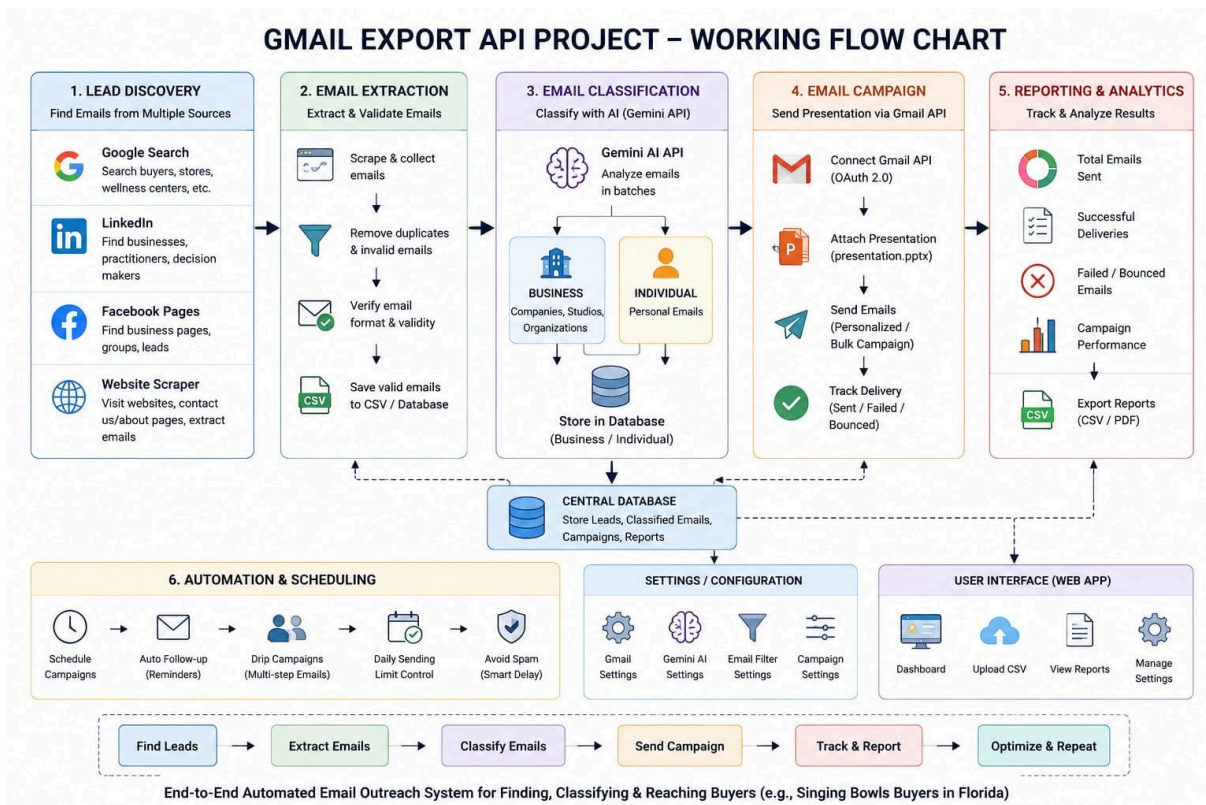


Figure 3 — Full pipeline: Lead Discovery -> Extraction -> Classification -> Campaign -> Reporting, with Automation, Settings, and UI subsystems

- Lead Discovery — expands source coverage beyond the five core channels to include general web scraping of business/contact pages.
- Email Classification — an AI classification stage (Section 5.4) sits between extraction and sending, splitting contacts into business and individual segments before dispatch.

- Reporting & Analytics — tracks total sent, successful deliveries, and failed/bounced emails, with CSV/PDF export.
- Automation & Scheduling — a forward-looking layer (see Section 14) for scheduled campaigns, automatic follow-ups, drip sequences, and send-rate control.
- Settings / Configuration and the web-based User Interface wrap the whole pipeline so it can be operated and tuned without touching code.

4. System Architecture

The system follows a modular pipeline architecture, where each stage is isolated into its own component to allow independent testing and future replacement (e.g., swapping the search backend or the mail transport).

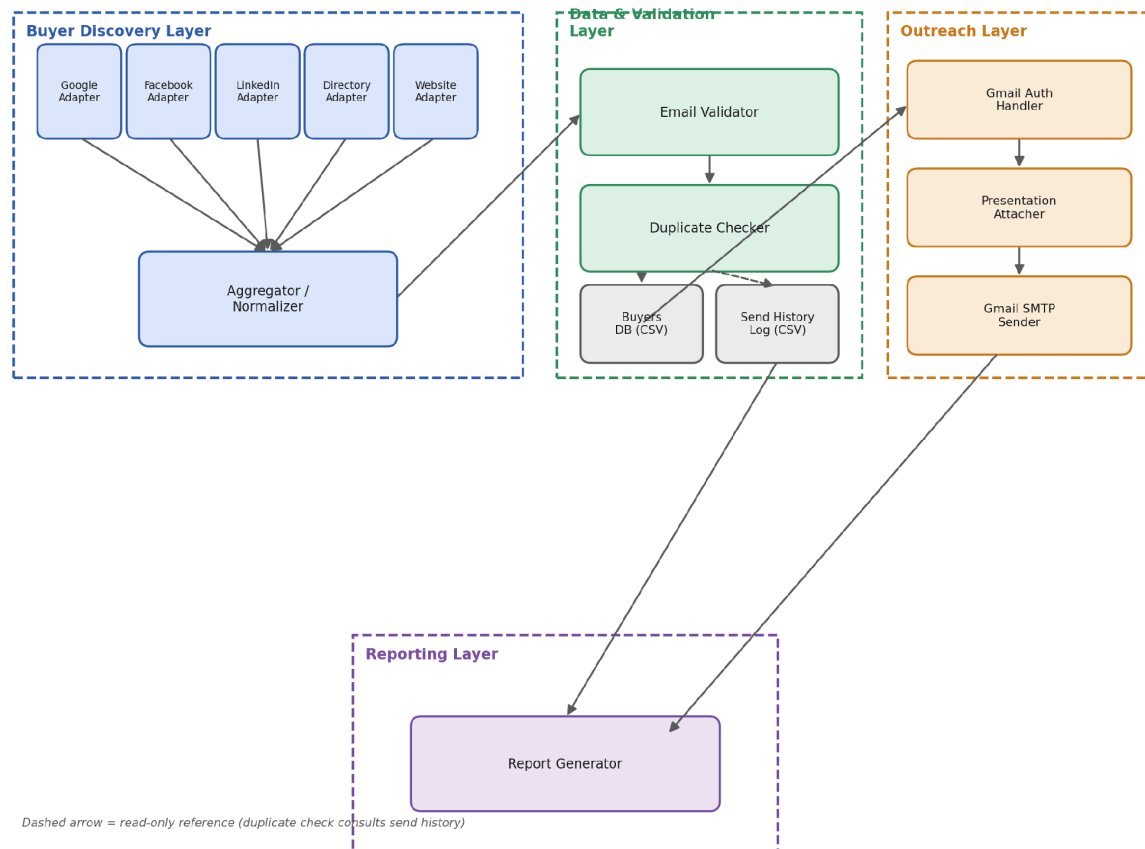


Figure 2 — Modular architecture: Discovery, Data & Validation, Outreach, and Reporting layers

Architectural Principles

- Separation of concerns — discovery, validation, dispatch, and reporting are independent modules connected only through the shared CSV data layer.

- Idempotent runs — the duplicate checker consults the send-history log before every dispatch, so re-running the discovery step never results in double-emailing a buyer.
- Source-adapter pattern — each platform (Google, Facebook, LinkedIn, directories, websites) is implemented as an interchangeable adapter feeding a common normalized schema, so new sources can be added without touching the outreach layer.

5. Functional Modules

5.1 Buyer Search Module

Responsible for querying each configured source with the target product keyword and buyer-intent qualifiers, and returning raw result sets (search snippets, post text, directory listings, or website contact pages).

- Iterates through configured platforms sequentially per run.
- Applies source-specific query construction (e.g., search operators for Google, directory category filters for business directories).
- Returns unstructured/semi-structured raw content to the Data Extraction module.

5.2 Data Extraction Module

Parses the raw content returned by the Buyer Search Module into the normalized buyer schema.

Field	Description
buyer_name	Name of the individual contact, where available
company_name	Company or business name associated with the buyer
email	Extracted contact email address
website	Company or listing website URL
country	Buyer's country, inferred from listing/profile/website data
source_platform	Origin channel (Google / Facebook / LinkedIn / Directory / Website / Other)

5.3 Email Validation Module

Filters extracted email addresses before they are allowed into the outreach queue.

- Applies regex-based syntax validation (local-part@domain.tld structure).
- Excludes obviously invalid, placeholder, or malformed addresses.
- Flags records with missing or unparseable emails for manual review rather than silent discard.

5.4 AI Email Classification Module

Segments validated contacts into business and individual categories using an AI language model (Gemini API) rather than hand-crafted heuristic rules, so the outreach message and audience selection can be tailored to each segment.

- Deduplicates the incoming contact list before classification to avoid redundant API calls.
- Sends addresses to the Gemini API in batches and reads back a business/individual label for each.
- Writes results into separate business and individual contact stores, which downstream feed the audience selector in the Gmail Sender Module.
- Chosen over a rule-based classifier because LLMs generalize well to this task with little or no task-specific training data ("zero/few-shot" classification).

5.5 Gmail Sender Module

Handles the actual dispatch of outreach emails through a Gmail account.

- Authenticates via SMTP using Gmail App Password credentials.
- Composes the outbound message with buyer-specific personalization (name/company where available), CC'ing a fixed monitoring address for visibility into sent content.
- Sends through Gmail's SMTP servers and captures the delivery outcome (sent / failed).
- Automatically reconnects and re-logs in if the SMTP session is dropped mid-campaign, then retries the current recipient before continuing.
- Applies a configurable delay between consecutive sends to reduce the risk of spam-rate throttling.

5.6 Presentation Attachment Module

Manages the company presentation file that is attached to every outreach email.

- Loads the presentation asset from a designated local directory.
- Attaches it to each outbound MIME message.
- Ensures consistent file naming and format on every send to avoid attachment-caching or "no name" issues.

5.7 Logging Module

Maintains a persistent, auditable record of both discovery and outreach activity.

- Appends every discovered buyer record to the buyers database.
- Appends every send attempt (recipient, timestamp, status) to the send-history log.
- Powers the duplicate-prevention check and the summary report.

6. Folder / File Structure

```
export-automation/
|
|— main.py           # Entry point – orchestrates search -> validate ->
send -> report
|— config.py        # Source list, search keywords, presentation path,
run settings
|
|— search/
```

└─ google_search.py	# Google source adapter
└─ facebook_search.py	# Facebook source adapter
└─ linkedin_search.py	# LinkedIn source adapter
└─ directory_search.py	# Business directory source adapter
└─ website_search.py	# Company website source adapter
└─ extraction/	
└─ data_extractor.py	# Normalizes raw results into the buyer schema
└─ validation/	
└─ email_validator.py	# Regex-based email validation logic
└─ outreach/	
└─ gmail_auth.py	# Gmail SMTP authentication handler
└─ gmail_sender.py	# Email composition & dispatch
└─ attachment_handler.py	# Presentation file attachment logic
└─ logging/	
└─ activity_logger.py	# Writes to buyers.csv and sent_log.csv
└─ reports/	
└─ report_generator.py	# Builds the run summary report
└─ assets/	
└─ company_presentation.pdf	# Attachment sent to every buyer
└─ data/	
└─ buyers.csv	# Discovered buyer records
└─ sent_log.csv	# Outreach history / duplicate-prevention source
└─ .env	# Gmail credentials & environment configuration
└─ requirements.txt	# Python dependencies

Key File Responsibilities

File	Purpose
main.py	Orchestrates the full pipeline: triggers each search adapter, hands results to extraction/validation, and initiates the outreach loop
config.py	Central configuration — target keyword(s), enabled sources, presentation file path, send limits
data_extractor.py	Converts raw per-source results into the common buyer record schema
email_validator.py	Validates and filters buyer emails before queuing
gmail_sender.py	Builds and sends the MIME email through Gmail SMTP
activity_logger.py	Single point of truth for all CSV read/write operations
buyers.csv	Local database of all discovered buyer leads

File	Purpose
sent_log.csv	Local database of all outreach attempts, used for de-duplication and reporting

7. Data Model & Database Schema

Although the system persists data in flat CSV/JSON files rather than a relational database, the underlying information can be modelled conceptually as four entities: BUYER_RECORD, CAMPAIGN, DELIVERY_RECORD, and SETTINGS. BUYER_RECORD and CAMPAIGN participate in a many-to-many relationship realized through the associative DELIVERY_RECORD entity, which captures the per-recipient, per-campaign delivery status.

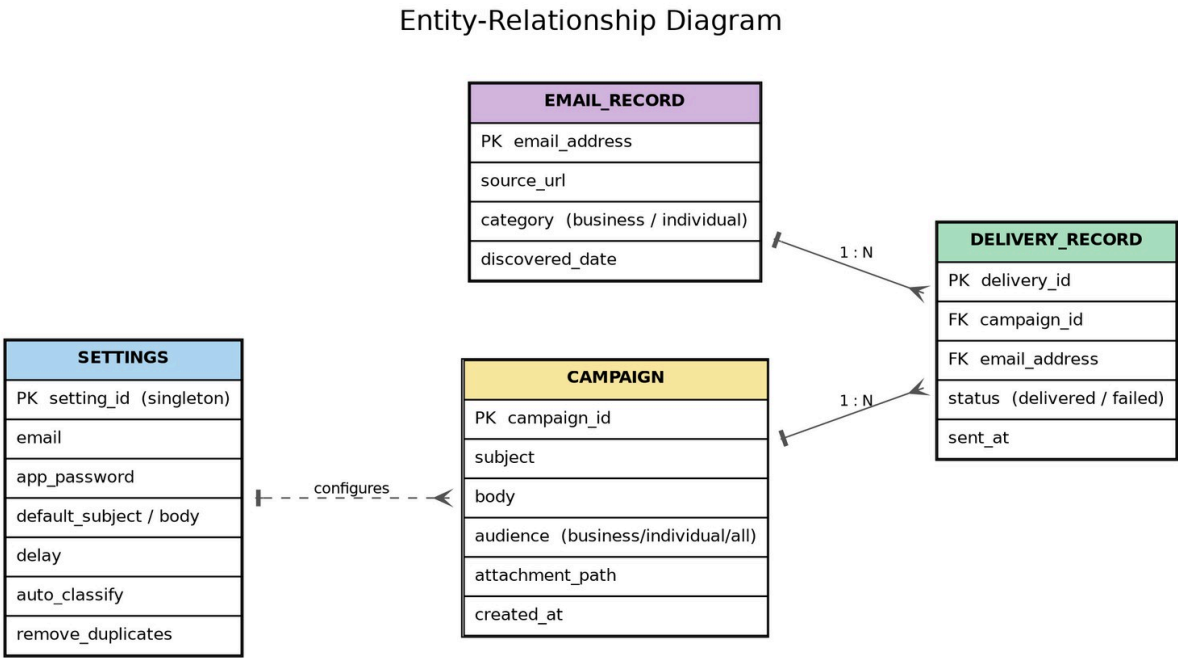


Figure 4 — Entity-Relationship Diagram (conceptual data model)

7.1 File-Based Schema

Each flat file below is documented as an equivalent table schema, which would map directly onto relational tables if the system were migrated to SQLite or PostgreSQL.

Store	Field	Description
buyers.csv	email_address (PK)	Unique discovered buyer email, one per row
buyers.csv	buyer_name / company_name	Extracted contact and company name
buyers.csv	website / country / source_platform	Supporting metadata captured at discovery time
business_emails.csv	email_address (PK)	Buyer emails classified as "business" by the AI Classification Module

Store	Field	Description
individual_emails.csv	email_address (PK)	Buyer emails classified as "individual"
sent_log.csv	email_address, status, timestamp	Delivery outcome ("sent" / "failed") for each send attempt
settings.json	email, app_password, delay, daily_send_limit	Singleton configuration document backing the Settings module

An in-memory `report_data` structure (`total`, `success_count`, `failed_count`, `successful[]`, `failed[]`) holds the results of the most recent campaign and is rebuilt each time a run completes, feeding both the on-screen report and the CSV export.

8. Application Interface (Web Routes)

The system exposes its functionality through a small set of browser-accessible routes, so an operator can run the full pipeline — search, review, classify, send, and report — without touching code.

Route	Description
/ (Home)	Landing dashboard summarizing recipients, campaigns, and delivery stats
/upload	Upload a CSV of leads, or view current database statistics (count, size, last modified)
/classify	Deduplicate contacts and run AI classification into business / individual segments
/send	Compose subject/body, choose audience (business / individual / all), attach the presentation, and launch the campaign
/report	View total sent, successful, and failed counts with computed success rate
/settings	Configure Gmail credentials, default subject/body, send delay, and classification preferences
/download-report	Stream the current campaign report as a downloadable CSV file

All routes render server-side HTML views; only the `download-report` endpoint streams a file directly.

9. Gmail Integration

9.1 Authentication

The system authenticates to Gmail using an App Password rather than the primary account password, in line with Google's security requirements for third-party SMTP clients.

- 2-Step Verification must be enabled on the sending Gmail account.
- An App Password is generated from Google Account → Security → App Passwords.
- Credentials are stored only in the local `.env` file and never hard-coded into source files.

9.2 SMTP Configuration

Setting	Value
SMTP Host	smtp.gmail.com
Port (STARTTLS)	587
Port (SSL, fallback)	465
Auth Method	App Password

9.3 Sending Process

10. Establish an SMTP connection to smtp.gmail.com.
11. Authenticate using the Gmail address and App Password from .env.
12. For each queued buyer record: compose the email body, attach the presentation, send, and capture the SMTP response.
13. Record the outcome in the send-history log.

9.4 Reliability Considerations

- A short delay is applied between consecutive sends to avoid Gmail's spam-rate throttling.
- Failed sends are logged with a failed status rather than silently dropped, allowing a retry pass.

10. Installation & Configuration

10.1 Requirements

- Python 3.10+
- A Gmail account with 2-Step Verification and an App Password
- Internet access to reach the configured search sources

10.2 Core Libraries

```
requests>=2.31.0
beautifulsoup4>=4.12.0
pandas>=2.0.0
python-dotenv>=1.0.0
validate-email>=1.3
```

10.3 Environment Variables (.env)

```
# --- Gmail Credentials (SMTP) ---
GMAIL_EMAIL=your_gmail_address@gmail.com
GMAIL_APP_PASSWORD=abcd efgh ijkl mnop

# --- Run Configuration ---
SEARCH_KEYWORD=Singing Bowls
```

```
DAILY_SEND_LIMIT=100
PRESENTATION_PATH=assets/company_presentation.pdf
```

10.4 Gmail Setup Steps

14. Enable 2-Step Verification on the Gmail account.
15. Navigate to Google Account → Security → App Passwords.
16. Generate an App Password for "Mail" / "Other (Custom name)".
17. Copy the generated 16-character password into GMAIL_APP_PASSWORD in .env.

11. Execution Guide

11.1 Step-by-Step Process

18. Configure .env with Gmail credentials and run parameters.
19. Confirm the company presentation file is present at PRESENTATION_PATH.
20. Run the pipeline: `python main.py`
21. The system executes buyer discovery across all configured sources.
22. Discovered leads are written to `data/buyers.csv`.
23. Valid, non-duplicate buyers are queued and emailed automatically.
24. Each send is recorded in `data/sent_log.csv`.
25. A run summary is printed / generated at completion.

11.2 Expected Output

Artifact	Description
<code>data/buyers.csv</code>	Updated with newly discovered buyer records for the run
<code>data/sent_log.csv</code>	Updated with one row per outreach attempt (sent/failed)
Console / report output	Summary counts: buyers found, emails validated, emails sent, duplicates skipped

12. Core Algorithms

The three pseudocode listings below describe the exact algorithmic behavior of the discovery, classification, and sending stages.

12.1 Algorithm — Email Extraction & Validation

```
INPUT: keyword, source list, max_results
OUTPUT: set of validated buyer records
```

```

1. records <- empty set
2. FOR each configured source:
3.   results <- query source with keyword
4.   FOR each candidate URL/post in results:
5.     IF NOT source-blocklisted: visit page, extract raw HTML/text
6.     candidates <- regex_match(EMAIL_PATTERN, raw_content)
7.     FOR each e in candidates:
8.       IF e ends with an image extension: discard
9.       ELSE IF domain part length > 50: discard
10.      ELSE add normalized record {name, company, e, website, country, source} to
records
11. WRITE records to buyers.csv
12. RETURN records

```

12.2 Algorithm — AI Email Classification

```

INPUT: buyers.csv (raw contact list)
OUTPUT: business_emails.csv, individual_emails.csv

1. emails <- unique_emails(buyers.csv)           // de-duplicate
2. batches <- split emails into manageable batches
3. FOR each batch:
4.   response <- call Gemini AI API(batch)
5.   FOR each (email, label) in response:
6.     IF label == business: append email to business_emails.csv
7.     ELSE: append email to individual_emails.csv

```

12.3 Algorithm — Send Email Campaign

```

INPUT: subject, body, audience, attachment (optional)
OUTPUT: success_count, failed_count, report_data

1. files <- select CSV(s) for audience (business / individual / both)
2. emails <- de-duplicated union of rows read from files
3. IF emails is empty: flag error; RETURN
4. IF attachment provided AND extension allowed: attach it
5. smtp <- SMTP_SSL connect + login(EMAIL, APP_PASS)
6. FOR each receiver in emails:
7.   TRY:
8.     build EmailMessage(subject, body, To=receiver, Cc=monitor, attachment)
9.     TRY: smtp.send_message(msg)
10.    CATCH SMTPServerDisconnected: reconnect + re-login; retry send
11.    success_count += 1; sleep(delay)
12.    CATCH Exception: failed_count += 1; log receiver as failed
13. smtp.quit()
14. report_data <- {total, success_count, failed_count, successful[], failed[]}

```

13. Error Handling & Limitations

Category	Handling Approach
Invalid email format	Record flagged and excluded from the send queue, retained in the dataset for review
Duplicate buyer	Skipped automatically via cross-reference with sent_log.csv
SMTP send failure	Logged with a failed status; run continues to the next buyer
Source unavailable / blocked	Source adapter fails gracefully; run proceeds with remaining sources
Missing presentation file	Run halts with a configuration error before any sends are attempted

Known Limitations

- Buyer data quality depends on the completeness of publicly available listings — some platforms yield partial records (e.g., missing direct emails).
- Country and company fields are inferred from unstructured text and may require manual correction in edge cases.
- The system is script-driven and requires manual initiation per run rather than continuous background operation.
- Search-source scraping depends on each platform's page structure and is fragile to markup changes or stronger bot detection.
- Gmail and AI-classification credentials are stored in local configuration files rather than a secrets manager — acceptable for single-operator use but a concern if the system is shared or deployed more broadly.
- CSV/JSON storage does not scale well to very large buyer lists and lacks the transactional guarantees, indexing, or concurrent-access safety of a real database.
- The current implementation does not include unsubscribe links, consent tracking, or explicit compliance workflows required by regulations such as CAN-SPAM, GDPR, or India's IT Act for production-scale commercial email marketing — this would need to be strengthened before commercial use.
- No authentication layer exists on the web interface itself, so anyone with network access to the app could trigger a campaign.

14. Future Improvements

- Scheduled automation — run discovery and outreach on a recurring schedule without manual initiation.
- AI-personalized messaging — generate buyer-specific email copy based on company/website context, similar to JD-based resume tailoring in other outreach systems.
- Reply tracking (IMAP sync) — detect and log buyer replies automatically to prioritize follow-ups.
- Centralized dashboard — replace CSV-based reporting with a lightweight analytics dashboard (buyers found, conversion rate, top-performing source).
- CRM export — allow one-click export of qualified buyer records into a CRM or spreadsheet-based sales tracker.

- Migrate CSV/JSON storage to a relational database (e.g., SQLite/PostgreSQL) using the ER model in Section 7, enabling richer querying, history, and multi-user support.
 - Introduce user authentication and role-based access control for the web interface.
 - Replace direct SMTP credential storage with OAuth 2.0 for Gmail API access, improving security and aligning with Google's recommended integration path.
 - Move long-running scraping and sending operations to background task queues (e.g., Celery/RQ) with real-time progress reporting instead of blocking the main request thread.
 - Add consent management and one-click unsubscribe handling to support regulatory compliance for commercial email campaigns.
 - Generalize the AI classification module beyond business/individual into finer-grained segments (e.g., industry, buyer intent, likely order size) for more precise targeting.
-